

## 1. Visão geral

O **Orçamento Preliminar IFC** é um aplicativo web que automatiza a geração de um orçamento preliminar de obra a partir de um modelo **BIM** no formato **IFC** (Industry Foundation Classes). O usuário importa um arquivo `.ifc`, visualiza o modelo em 3D e obtém uma planilha orçamentária com serviços, quantidades, unidades e preços — exportável em Excel.

A premissa central é que o IFC, por ser um dado **estruturado**, já carrega as quantidades calculadas dos elementos construtivos ( `BaseQuantities` ). Isso elimina a calibração manual e os erros de medição típicos da extração a partir de plantas em PDF.

## 2. Objetivos

- Reduzir o tempo de levantamento de quantitativos (takeoff) de obras.
- Traduzir elementos BIM em **linguagem de orçamento** (serviço/unidade/quantidade).
- Precificar automaticamente com base em referência **SINAPI**, permitindo ajustes manuais pelo orçamentista.
- Entregar uma planilha pronta para uso em `.xlsx`.

## 3. Arquitetura e stack

| Camada           | Tecnologia                                      | Responsabilidade                               |
|------------------|-------------------------------------------------|------------------------------------------------|
| Front-end / UI   | Next.js (App Router) + React + Tailwind         | Upload, viewer 3D, tabela editável, exportação |
| Visualização 3D  | web-ifc                                         | Renderiza o modelo e destaca objetos           |
| Parsing          | Parser STEP próprio ( <code>lib/ifc.ts</code> ) | Lê elementos IFC e quantidades                 |
| Regra de negócio | <code>lib/budget.ts</code>                      | Traduz elementos em itens de orçamento         |
| Precificação     | <code>lib/sinapi.ts</code>                      | Tabela de referência de custos                 |
| Exportação       | <code>exceljs ( lib/ifcXlsx.ts )</code>         | Gera a planilha <code>.xlsx</code>             |

### 3.1 Estrutura de pastas

|                                          |                                                    |
|------------------------------------------|----------------------------------------------------|
| <code>app/page.tsx</code>                | # UI: upload IFC, viewer 3D, tabela e download     |
| <code>app/layout.tsx</code>              | # metadados e fontes                               |
| <code>app/api/ifc/route.ts</code>        | # endpoint: parse do IFC -> orçamento (JSON)       |
| <code>app/api/ifc/export/route.ts</code> | # endpoint: gera a planilha <code>.xlsx</code>     |
| <code>lib/ifc.ts</code>                  | # parser STEP + extração de quantidades + unidades |
| <code>lib/budget.ts</code>               | # tradução para itens de orçamento (PT-BR)         |
| <code>lib/sinapi.ts</code>               | # tabela de referência de custos SINAPI            |
| <code>lib/ifcXlsx.ts</code>              | # montagem da planilha (exceljs)                   |

## 4. Fluxo de dados

1. **Upload** — o usuário seleciona um `.ifc` na interface.
2. **Parse** ( `POST /api/ifc` ) — o arquivo é lido pelo parser STEP, que extrai os elementos (paredes, lajes, pilares, vigas, fundações, portas, janelas, coberturas) e suas quantidades.
3. **Orçamentação** ( `lib/budget.ts` ) — os elementos viram itens de serviço agrupados por etapa, com unidade e quantidade.
4. **Precificação** ( `lib/sinapi.ts` ) — cada item recebe um preço unitário de referência SINAPI (ou inserido pelo usuário).
5. **Visualização** — a tabela é exibida na web; o modelo aparece em 3D.

6. **Exportação** ( `POST /api/ifc/export` ) — gera o `.xlsx` com os preços efetivos.

## 5. Parser IFC ( `lib/ifc.ts` )

O IFC segue o padrão **STEP / ISO-10303-21**. O parser implementado é leve e não usa WASM. Principais responsabilidades:

- **Leitura das entidades** ( `#id = IFCTIPO(args)` ) e dos conjuntos de quantidades ( `IfcElementQuantity / BaseQuantities` ).
- **Normalização de unidades**: comprimentos  $\text{mm} \rightarrow \text{m}$ , áreas  $\text{mm}^2 \rightarrow \text{m}^2$ , volumes  $\text{mm}^3 \rightarrow \text{m}^3$ , com heurística por magnitude do valor.
- **Deduplicação** de quantidades repetidas pelo exportador (mantém o valor líquido / mínimo coerente).
- **Fallback geométrico**: quando um elemento não tem `BaseQuantities`, tenta derivar a quantidade a partir da geometria (ex.: `IfcExtrudedAreaSolid + IfcRectangleProfileDef` ).

## 6. Regras de orçamento ( `lib/budget.ts` )

Os elementos IFC são convertidos em itens agrupados por etapa. Resumo:

| Etapas                   | Itens (exemplos)                                  | Unidade                              |
|--------------------------|---------------------------------------------------|--------------------------------------|
| 1. Serviços preliminares | Limpeza do terreno, locação da obra               | m <sup>2</sup> , m                   |
| 2. Fundações             | Concreto, fôrmas, armadura                        | m <sup>3</sup> , m <sup>2</sup> , kg |
| 3. Superestrutura        | Concreto em pilares/vigas/lajes, fôrmas, armadura | m <sup>3</sup> , m <sup>2</sup> , kg |
| 4. Alvenaria/Vedações    | Paredes externas e internas                       | m <sup>2</sup>                       |
| 5. Cobertura             | Telhamento (inclui estrutura)                     | m <sup>2</sup>                       |
| 6. Esquadrias            | Portas, janelas e seus vãos                       | un, m <sup>2</sup>                   |
| 7. Revestimentos         | Chapisco, emboço, reboco                          | m <sup>2</sup>                       |
| 8. Pisos                 | Contrapiso, revestimento de piso                  | m <sup>2</sup>                       |
| 9. Pintura               | Paredes e teto/forro                              | m <sup>2</sup>                       |

Critérios de engenharia adotados:

- **Paredes** classificadas em externa/interna por palavra-chave do nome; área de revestimento considera 1 face para externas e 2 faces para internas.
- **Cobertura** usa a **face inferior** das lajes de telhado (forro) — não a superfície total da laje (que somaria faces e bordas).
- **Pintura de teto** só aparece quando existe laje de teto no modelo.
- Itens marcados como **(estimado)** derivam de premissas paramétricas (ex.: ~10 m<sup>2</sup> de fôrma por m<sup>3</sup> de concreto, ~100 kg/m<sup>3</sup> de armadura).

## 7. Precificação SINAPI ( `lib/sinapi.ts` )

O app oferece uma **Fonte de Custos** selecionável no topo:

- **SINAPI (padrão)** — preços de referência (estado de **SP**, regime **não desonerado**) alimentam a coluna de preço unitário. Cada item do orçamento é mapeado para a composição SINAPI mais próxima, garantindo **unidade compatível**. A interface exibe o código/descrição da composição usada.
- **Inserido pelo Usuário** — todos os preços são digitados manualmente.

No modo SINAPI, o usuário ainda pode **editar** o preço de um item específico (que então deixa de mostrar o código SINAPI); o botão  **SINAPI** devolve aquele item ao valor de referência.

Os valores SINAPI embutidos são **referências aproximadas**. O SINAPI oficial é publicado mensalmente pela Caixa por UF; recomenda-se revisar os números de `lib/sinapi.ts` conforme o mês/estado do orçamento.  
<https://www.caixa.gov.br/site/Paginas/downloads.aspx>

## 8. Exportação Excel ( `lib/ifcXlsx.ts` )

A planilha `.xlsx` é montada com **exceljs** e contém:

- **Orçamento** — código, etapa, serviço, unidade, quantidade, preço unitário, total e critério/observação, com linha de total geral.
- **Detalhe** — lista por elemento (tipo, nome, GUID, área, comprimento, volume).

## 9. Visualizador 3D

O modelo IFC é renderizado com **web-ifc**. Cada item do orçamento pode **destacar** no modelo os objetos paramétricos que originaram sua quantidade, facilitando a conferência visual do takeoff.

## 10. Como executar

```
npm install
npm run dev      # http://localhost:3000
npm run build
npm run typecheck
```

Requer Node.js 18+ (testado com Node 24).

### Uso

1. Abra `http://localhost:3000`.
2. Clique em **Abrir IFC** e selecione um arquivo `.ifc`.
3. Confira o modelo 3D e a planilha; ajuste a **Fonte de Custos** e os preços.
4. Clique em **Exportar Excel**.

## 11. Validação

Testado com o modelo **BasicHouse.ifc** (IFC2X3 exportado do Revit). As quantidades e os preços foram conferidos item a item, validando a compatibilidade de unidades entre o app e a referência SINAPI.

## 12. Limitações e próximos passos

- Quantificação **por elemento**, não por ambiente (o IFC de teste não traz `IfcSpace`). Suporte a área por cômodo pode ser adicionado quando houver espaços no modelo.
- Classificação externa/interna de paredes por **palavra-chave** do nome.
- Referência SINAPI **estática** e regional (SP) — evoluir para múltiplas UFs e seleção do mês de referência.
- Possível adicionar BDI, encargos e composição detalhada de preços.